

**INTERNATIONAL CYBERSECURITY AND
DIGITAL FORENSICS ACADEMY**

Week 5 Practical Laboratory

Simulating and Analyzing Security Governance Scenarios

BY

MIRACLE NWADIARU

C11/26/CGRCE/17587

Part 1: Setting Up a Simulated Environment

1.1 Docker environment setup

```
(icdfa@icdfa)~$ sudo systemctl enable --now docker
Synchronizing state of docker.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable docker

(icdfa@icdfa)~$ docker --version
Docker version 28.5.2-dfsg4, build 9cc0dea35e9a963f281434761c656fba4ac43aed

(icdfa@icdfa)~$ sudo systemctl status docker --no-pager
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
   Active: active (running) since Tue 2026-10-06 11:09:45 WAT; 1min 42s ago
     Invocation: 22513dffb5e04e88af59a2b4d43b67a9
   TriggeredBy: ● docker.socket
    Docs: https://docs.docker.com
   Main PID: 36980 (dockerd)
     Tasks: 8
    Memory: 25.6M (peak: 28M)
       CPU: 651ms
    CGroup: /system.slice/docker.service
            └─36980 /usr/sbin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock

Oct 06 11:09:43 icdfa dockerd[36980]: time="2026-10-06T11:09:43.605915715+01:00" level=info msg="CDI directory does not exist, skipping: failed to monitor for changes: no such file or directory" dir=/var/run/cdi
Oct 06 11:09:43 icdfa dockerd[36980]: time="2026-10-06T11:09:43.697373606+01:00" level=info msg="Creating a containerd client" address=/run/containerd/containerd.sock timeout=1m0s
Oct 06 11:09:43 icdfa dockerd[36980]: time="2026-10-06T11:09:43.894604652+01:00" level=info msg="Loading containers: start."
Oct 06 11:09:44 icdfa dockerd[36980]: time="2026-10-06T11:09:44.787197534+01:00" level=info msg="Loading containers: done."
Oct 06 11:09:44 icdfa dockerd[36980]: time="2026-10-06T11:09:44.878420929+01:00" level=info msg="Docker daemon" commit=3ef3c07c883be49a2e8097bcfd2952fce1d70eb containerd-snapshotter=false storage-d_rson=28.5.2-dfsg4
Oct 06 11:09:44 icdfa dockerd[36980]: time="2026-10-06T11:09:44.892915359+01:00" level=info msg="Initializing buildkit"
Oct 06 11:09:45 icdfa dockerd[36980]: time="2026-10-06T11:09:45.001337851+01:00" level=info msg="Completed buildkit initialization"
Oct 06 11:09:45 icdfa dockerd[36980]: time="2026-10-06T11:09:45.005553826+01:00" level=info msg="Daemon has completed initialization"
Oct 06 11:09:45 icdfa systemd[1]: Started docker.service - Docker Application Container Engine.
Oct 06 11:09:45 icdfa systemd[1]: API listen on /run/docker.sock"
Hint: Some lines were ellipsized, use -l to show in full.
```

Fig 1: Docker Installation Verification

Fig 1 confirms Docker is installed correctly and available for creating and managing the laboratory containers.

1.2. Docker compose verification

```
(icdfa@icdfa)~$ docker-compose config
NAME[0000] /home/icdfa/security_governance_lab/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to :
name: security_governance_lab
services:
  database:
    container_name: database_server
    environment:
      MYSQL_DATABASE: customer_data
      MYSQL_PASSWORD: app_password
      MYSQL_ROOT_PASSWORD: password
      MYSQL_USER: app_user
    image: mysql:5.7
    networks:
      secgov_network: null
    volumes:
      - type: bind
        source: /home/icdfa/security_governance_lab/database_init
        target: /docker-entrypoint-initdb.d
        bind:
          create_host_path: true
    monitoring:
      command:
        - tail
        - -f
        - /dev/null
      container_name: monitoring_server
      image: ubuntu:20.04
      networks:
        secgov_network: null
      volumes:
        - type: bind
          source: /home/icdfa/security_governance_lab/monitoring
          target: /monitoring
          bind:
            create_host_path: true
  web:
    container_name: web_server
    image: vulhub/struts2:2.3.30
    networks:
      secgov_network: null
    ports:
      - mode: ingress
        target: 8080
        published: "8080"
        protocol: tcp
networks:
  secgov_network: null
```

Fig 2: Docker Compose Verification

Fig 2: Shows that the Docker Compose configuration ran successfully.

1.3. Lab Directory Setup

```
(icdfa@icdfa)-[~]
$ mkdir -p ~/security_governance_lab
$ cd ~/security_governance_lab
$ pwd
/home/icdfa/security_governance_lab
$ ls -la
total 8
drwxrwxr-x  2 icdfa icdfa 4096 Oct  6 13:39 .
drwx----- 28 icdfa icdfa 4096 Oct  6 13:39 ..
$
```

Fig 3: Lab Directory setup

Fig 3: Shows the creation and organization of the security governance laboratory working directory and required project files.

1.4. Security Network Creation

```
(icdfa@icdfa)-[~/security_governance_lab]
$ docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
2d5851643beb        bridge              bridge              local
160da1d9cc64        host                host                local
3022114e0849        none                null                local
fa39fb90c68c        secgov_network      bridge              local

(icdfa@icdfa)-[~/security_governance_lab]
$ docker network inspect secgov_network
[
  {
    "Name": "secgov_network",
    "Id": "fa39fb90c68cc1e4108a2c116be511db9a812be7aa7de9e0929f25025a2627c0",
    "Created": "2026-10-06T13:42:29.741987235+01:00",
    "Scope": "local",
    "Driver": "bridge",
    "EnableIPv4": true,
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": {},
      "Config": [
        {
          "Subnet": "172.18.0.0/16",
          "Gateway": "172.18.0.1"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    },
    "ConfigOnly": false,
    "Containers": {},
    "Options": {},
    "Labels": {}
  }
]
```

Fig 4: Security Network Creation

Fig 4: Confirms the external secgov_network was created successfully for communication between laboratory containers.

1.5. Database Initialization

```
(icdfa@icdfa)-[~/security_governance_lab]
$ nano database_init/init.sql

(icdfa@icdfa)-[~/security_governance_lab]
$ cat database_init/init.sql
CREATE DATABASE IF NOT EXISTS customer_data;
USE customer_data;

CREATE TABLE customers (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100),
  ssn VARCHAR(20),
  dob DATE,
  address VARCHAR(255),
  email VARCHAR(100),
  phone VARCHAR(20),
  credit_card VARCHAR(20)
);

INSERT INTO customers
(name, ssn, dob, address, email, phone, credit_card)
VALUES
('John Doe', '000-00-0001', '1985-05-15',
'123 Example Street', 'john.doe@example.local',
'08000000001', '4111111111111111'),

('Jane Smith', '000-00-0002', '1990-08-20',
'456 Sample Avenue', 'jane.smith@example.local',
'08000000002', '4222222222222222'),

('Robert Brown', '000-00-0003', '1982-11-10',
'789 Test Road', 'robert.brown@example.local',
'08000000003', '4333333333333333');

(icdfa@icdfa)-[~/security_governance_lab]
$

(icdfa@icdfa)-[~/security_governance_lab]
$ docker exec database_server mysql -uapp_user -papp_password -D customer_data -e "SELECT id,name,email FROM customers;"
mysql: [Warning] Using a password on the command line interface can be insecure.
id  name  email
1   John Doe  john.doe@example.local
2   Jane Smith  jane.smith@example.local
3   Robert Brown  robert.brown@example.local

(icdfa@icdfa)-[~/security_governance_lab]
$
```

Fig 5: Database Initialization & Output

Fig 5: Confirms successful MySQL container initialization and verifies that the customer database and required configuration were correctly created using the supplied initialization files.

1.6. Lab Containers running

```
(icdfa@icdfa)-[~/security_governance_lab]
$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
8af62285d789   ubuntu:20.04   "tail -f /dev/null"     About a minute ago Up About a minute                               monitoring_server
a5bb6586a19a   vulhub/struts2:2.3.30 "/usr/local/bin/mvn-"   About a minute ago Up About a minute  0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp  web_server
f279653984e9   mysql:5.7      "docker-entrypoint.s..." About a minute ago Up About a minute  3306/tcp, 33060/tcp                     database_server

(icdfa@icdfa)-[~/security_governance_lab]
$
```

Fig 6: Lab Containers running

1.7 Monitoring container verification

```
(icdfa@icdfa)-[~/security_governance_lab]
$ docker exec monitoring_server ls -la /monitoring
total 8
drwxrwxr-x 2 1000 1000 4096 Oct  6 12:54 .
drwxr-xr-x 1 root root 4096 Oct  6 14:32 ..

(icdfa@icdfa)-[~/security_governance_lab]
$ docker exec monitoring_server bash -c "echo 'Monitoring container operational' && uname -a"
Monitoring container operational
Linux 8af62285d789 6.12.13-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.12.13-1kali1 (2025-02-11) x86_64 x86_64 x86_64 GNU/Linux

(icdfa@icdfa)-[~/security_governance_lab]
$
```

Fig 7: Monitoring container verification

1.8. Patch Mangement List

```
(icdfa@icdfa)-[~/security_governance_lab]
$ ./patch_management.py --list
Available patches:
-----
ID: PATCH-001
Name: Apache Struts2 S2-045 Patch
Description: Fixes CVE-2017-5638 in Apache Struts2
Severity: Critical
Affected System: web_server
Status: Available

ID: PATCH-002
Name: MySQL Security Configuration
Description: Secures MySQL with proper authentication and access controls
Severity: High
Affected System: database_server
Status: Available

ID: PATCH-003
Name: Network Segmentation Rules
Description: Implements proper network segmentation between servers
Severity: Medium
Affected System: all
Status: Available

(icdfa@icdfa)-[~/security_governance_lab]
$
```

Fig 8: Monitoring container verification

Fig 8: Displays available security patches and their severity, demonstrating the organization's baseline patch management process.

1.9 Vulnerability Report

```
(icdfa@icdfa)-[~/security_governance_lab]
$ python3 -m json.tool "$(ls -t vulnerability_report_*.json | head -1)"
{
  "scan_date": "2026-10-07T09:36:27.740217",
  "vulnerabilities": [
    {
      "vulnerable": true,
      "vulnerability": "Apache Struts2 S2-045 (CVE-2017-5638)",
      "severity": "Critical",
      "description": "Remote Code Execution vulnerability in Apache Struts2",
      "remediation": "Update to Struts 2.3.32 or Struts 2.5.10.1",
      "reference": "https://nvd.nist.gov/vuln/detail/CVE-2017-5638"
    },
    {
      "vulnerable": true,
      "vulnerability": "MySQL using default credentials",
      "severity": "High",
      "description": "MySQL 5.7.44 accessible with default credentials",
      "remediation": "Change default credentials and restrict access",
      "reference": "https://dev.mysql.com/doc/refman/5.7/en/security.html"
    },
    {
      "vulnerable": true,
      "vulnerability": "Insufficient network segmentation",
      "severity": "Medium",
      "description": "Web server can directly access database server",
      "remediation": "Implement network segmentation and access controls",
      "reference": "https://www.cisecurity.org/controls/implement-network-segmentation"
    }
  ],
  "summary": {
    "total": 3,
    "critical": 1,
    "high": 1,
    "medium": 1,
    "low": 0,
    "unknown": 0
  }
}

(icdfa@icdfa)-[~/security_governance_lab]
$ ./vulnerability_scanner.py
Vulnerability scan completed. Report saved to vulnerability_report_20261007_090142.json
Summary: 3 vulnerabilities found
Critical: 0
High: 0
Medium: 1
Low: 0
Unknown: 1

(icdfa@icdfa)-[~/security_governance_lab]
$ ls -lh vulnerability_report_*.json
-rw-rw-r-- 1 icdfa icdfa 1.4K Oct  7 09:01 vulnerability_report_20261007_090142.json
```

Fig 9: Vulnerability Report

Fig 8: Shows the vulnerability scanner identifying critical, high, and medium security weaknesses within the simulated environment.

[Kindly click the link here](#) to find the “vulnerability scan report”

1.10. Governance Tracker Dashboard

```
(icdfa@icdfa)~/security_governance_lab
$ ./governance_tracker.py --dashboard
{
  "generated_date": "2026-10-07T08:53:31.707710",
  "summary": {
    "policies": 3,
    "controls": 3,
    "risks": 3,
    "incidents": 1,
    "metrics": 3,
    "audits": 1
  },
  "risk_summary": {
    "critical": 1,
    "high": 1,
    "medium": 1,
    "low": 0,
    "very_low": 0
  },
  "incident_summary": {
    "critical": 0,
    "high": 1,
    "medium": 0,
    "low": 0
  },
  "metrics_summary": [
    {
      "name": "Patch Compliance",
      "target": 95.0,
      "actual": 85.0,
      "status": "Red"
    },
    {
      "name": "Vulnerability Remediation Time",
      "target": 7.0,
      "actual": 12.0,
      "status": "Green"
    },
    {
      "name": "Security Incidents",
      "target": 0.0,
      "actual": 1.0,
      "status": "Green"
    }
  ]
}
```

Fig10: Governance Tracker Dashboard

Fig 10: Displays the governance tracker containing policies, controls, risks, incidents, metrics, and audit information.

[Kindly click the link here](#) to find the **“Governance Tracker dashboard”**

Governance Interpretation

The simulated environment recreated several governance conditions associated with major data breaches such as Equifax. A known critical vulnerability remained present even though vulnerability management and patch-management processes existed. This demonstrates the difference between having a policy or process on paper and actually enforcing it. Weak database configuration and insufficient segmentation also increased the potential impact of a successful

compromise. The exercise shows why governance must include clear ownership, measurable remediation targets, monitoring, verification and management oversight.

Part 2: Analyzing Security Governance Failures

2.1. The Attack Stimulation Log

```
(icdfa@icdfa)-[~/security_governance_lab]
$ python3 simulate_attack.py
Simulating attack on vulnerable Struts application...
System is vulnerable to Struts S2-045 (CVE-2017-5638)
Simulating data exfiltration...
Simulating access to database...
Simulating exfiltration of customer data...
Attack simulation completed. Log saved to attack_log.json

(icdfa@icdfa)-[~/security_governance_lab]
$
```

Fig 2.1 : The Attack Stimulation

Fig 2.1: Shows the controlled Struts2 attack successfully accessing the simulated customer database environment.

[Kindly click the link here](#) to find the **attack stimulation log**.

2.2 Governance Analysis Failures

```
(icdfa@icdfa)-[~/security_governance_lab]
$ ./analyze_governance_failures.py

=====
SECURITY GOVERNANCE FAILURE ANALYSIS
=====

Loaded vulnerability report: vulnerability_report_20261007_093627.json
Loaded attack simulation log
Loaded governance data

Governance failure analysis completed.
Total governance failures identified: 7
Report saved to: governance_failure_report_20261007_100409.json

Governance Failures:
1. Patch Management - High
   Critical and high severity vulnerabilities remain unpatched.
2. Incident Detection - High
   A successful attack was simulated without evidence of timely detection.
3. Policy Implementation - High
   A patch management policy exists, but the critical Struts vulnerability remains unpatched.
4. Vulnerability Management - High
   A vulnerability management policy exists while critical vulnerabilities remain unresolved.
5. Metrics and Measurement - Medium
   Patch compliance is measured, but critical vulnerabilities remain unresolved.
6. Security Architecture - High
   Web and database systems are not adequately segmented.
7. Authentication and Access Control - Medium
   Database security configuration has not been hardened.

(icdfa@icdfa)-[~/security_governance_lab]
$
```

Fig 2.2 : Governance Analysis Failures

Fig 2.2 Shows the analysis identifying seven governance failures across patch management, monitoring, policy implementation, and security architecture.

[Kindly click the link here](#) to find the **Governance analysis failure report**.

2.3. Comparison report between the simulated environment and the Equifax case study.

```
File Actions Edit View Help
~$ ls -lh equifax_comparison.md
-rw-rw-r-- 1 lcdfa lcdfa 5.2K Oct 7 10:09 equifax_comparison.md
~$ cat equifax_comparison.md
# Equifax Breach and Security Governance Comparison

## Overview
The Equifax data breach of 2017 demonstrated how weaknesses in security governance can allow a known technical vulnerability to develop into a major cybersecurity incident. Attackers exploited a vulnerability in Apache Struts that had a security patch available. The incident resulted in the exposure of sensitive personal information belonging to approximately 147 million people.

This laboratory provides a simplified simulation of several governance weaknesses associated with the Equifax breach. The simulated environment contains a vulnerable Apache Struts2 application, inadequate patch management, insufficient network segmentation, weak database security, and security governance controls that exist but are not effectively implemented.

## Key Governance Failures in the Equifax Breach

### 1. Patch Management
Equifax failed to ensure that the available Apache Struts security patch was consistently identified, applied, and verified. The simulated environment demonstrates the same type of failure because the vulnerable Struts application remains unpatched despite the vulnerability scanner identifying it as a Critical vulnerability.

### 2. Security Monitoring
Effective security monitoring should identify suspicious activity and provide timely alerts. In the simulated environment, the Struts attack succeeds and customer-data access is simulated without evidence of timely detection. This demonstrates the importance of continuous monitoring, centralized logging, intrusion detection, and incident alerting.

### 3. Network Segmentation
Network segmentation limits the ability of an attacker to move from a compromised public-facing system to sensitive internal systems. The laboratory allows the web server and database server to communicate directly, demonstrating inadequate segmentation and increasing the potential impact of a web-server compromise.

### 4. Leadership and Accountability
Security governance requires clear ownership, accountability, and escalation when security controls fail. A policy alone is insufficient if responsible personnel do not ensure that identified vulnerabilities are remediated. The governance tracker demonstrates how security policies, controls, risks, metrics, and audits should provide accountability.

### 5. Policy Implementation
Organizations may have security policies and procedures but still remain vulnerable when those policies are not effectively implemented. The laboratory contains a Patch Management Policy and Vulnerability Management Policy while the critical Struts vulnerability remains unresolved. This demonstrates the difference between having a policy and enforcing it.

## Comparison of Governance Failures

| Governance Area | Equifax | Simulated Environment | Severity |
|---|---|---|---|
| Patch Management | Known Struts vulnerability was not effectively patched | Critical Struts vulnerability remains unpatched | High |

## Comparison of Governance Failures

| Governance Area | Equifax | Simulated Environment | Severity |
|---|---|---|---|
| Patch Management | Known Struts vulnerability was not effectively patched | Critical Struts vulnerability remains unpatched | High |
| Security Monitoring | Attack activity was not detected quickly enough | Successful simulated attack without timely detection | High |
| Network Segmentation | Insufficient segmentation increased potential impact | Web and database systems are not adequately segmented | High |
| Authentication | Weak security controls contributed to exposure risk | MySQL security configuration remains unhardened | Medium |
| Policy Implementation | Security policies and procedures were not effectively enforced | Policies exist but critical vulnerabilities remain unresolved | High |
| Metrics and Measurement | Security processes were not sufficiently effective in measuring and driving remediation | Patch compliance is measured but critical issues remain open | Medium |

## Lessons Learned
The Equifax incident demonstrates that cybersecurity governance must connect technical security controls with management accountability. Identifying vulnerabilities is not enough; organizations must ensure that vulnerabilities are assigned to responsible personnel, remediated within defined timeframes, verified after remediation, and reported through meaningful security metrics.

The incident also demonstrates the importance of defense in depth. Patch management, vulnerability scanning, network segmentation, authentication controls, monitoring, incident response, and governance policies must work together. If one control fails, other controls should limit the attacker's ability to compromise additional systems or access sensitive information.

## Conclusion
The simulated environment demonstrates how multiple governance weaknesses can combine to increase the likelihood and impact of a major security incident. The vulnerable Struts application represents an initial point of compromise, while inadequate patch management, network segmentation, authentication, monitoring, policy implementation, and measurement increase the potential consequences.

The key lesson from both the Equifax breach and this laboratory is that effective cybersecurity requires continuous governance, accountability, monitoring, and verification. Organizations must not only establish security policies but also ensure that those policies produce measurable and consistently enforced security outcomes.
```

Fig 2.3: the simulated environment and the Equifax

Fig 2.3: Compares simulated governance weaknesses with lessons from the Equifax breach and highlights common control failures.

[Kindly click the link here](#) to find “**The comparison report between the simulated environment and the Equifax case study**”.

Key Governance Lessons

The Equifax case demonstrates that security policies are not effective unless organisations ensure that they are implemented, monitored and enforced. The lab showed that a critical vulnerability can remain exploitable when remediation is delayed, even when a patch-management process exists. Effective governance therefore requires assigned responsibility, remediation deadlines, verification and escalation when critical risks remain unresolved.

The exercise also demonstrated that security must rely on multiple layers. Stronger segmentation, secure database configuration, effective monitoring and governance oversight can reduce the likelihood that compromise of a public-facing application becomes a wider data-security incident. Overall, governance should be treated as an ongoing process involving identification, action, measurement and review.

Part 3: Implementing Security Governance Controls

3.1 The governance control implementation



```
(icdfa@icdfa)-[~/security_governance_lab]
$ ./implement_governance_controls.py --all
Implementing patch management controls...
Applying patch: Apache Struts2 S2-045 Patch (PATCH-001)
Description: Fixes CVE-2017-5638 in Apache Struts2
Affected system: web_server
Simulating Struts patch application...
Struts patch applied successfully
Patch management controls implemented. Control ID: CTL-004, Metric ID: MET-004
Implementing network segmentation controls...
Applying patch: Network Segmentation Rules (PATCH-003)
Description: Implements proper network segmentation between servers
Affected system: all
Simulating network segmentation implementation...
Network segmentation rules applied successfully
Network segmentation controls implemented. Control ID: CTL-005
Implementing database security controls...
Applying patch: MySQL Security Configuration (PATCH-002)
Description: Secures MySQL with proper authentication and access controls
Affected system: database_server
Simulating MySQL security configuration...
MySQL security configuration applied successfully
Database security controls implemented. Control ID: CTL-006
Implementing security monitoring controls...
Security monitoring controls implemented. Control ID: CTL-007, Metric ID: MET-005
Implementing governance oversight controls...
Governance oversight controls implemented. Policy ID: POL-004, Control ID: CTL-008
Verifying security controls...
Struts patched: False
MySQL secured: True
Network segmented: False
Security monitoring script implemented
Governance oversight document implemented
Policies: 4
Controls: 8
Metrics: 5
Vulnerability scan completed. Report saved to vulnerability_report_20261007_111204.json
Summary: 3 vulnerabilities found
Critical: 1
High: 1
Medium: 1
Low: 0
Unknown: 0
(icdfa@icdfa)-[~/security_governance_lab]
$
```

Fig 3.1: The governance control implementation

Fig 3.1: Demonstrates implementation of patch management, network segmentation, database security, monitoring, and governance oversight controls.

[Kindly click the link here](#) to find the “**The governance control implementation script**”

3.2 The governance controls report

```
(icdfa@icdfa)-[~/security_governance_lab]
$ cd ~/security_governance_lab
nano governance_controls_report.md

(icdfa@icdfa)-[~/security_governance_lab]
$ ls -l governance_controls_report.md
-rw-rw-r-- 1 icdfa icdfa 3364 Oct  7 11:30 governance_controls_report.md

(icdfa@icdfa)-[~/security_governance_lab]
$
```

Fig 3.2: Governance controls report

Fig 3.2: Shows the completed report documenting the security governance controls implemented within the laboratory.

[Kindly click the link here](#) to find the “Governance Controls Report”.

3.3 Screenshots showing the verification of implemented controls

```
(icdfa@icdfa)-[~/security_governance_lab]
$ cd ~/security_governance_lab
./implement_governance_controls.py --verify
Verifying security controls...
Struts patched: False
MySQL secured: True
Network segmented: False
Security monitoring script implemented
Governance oversight document implemented
Policies: 4
Controls: 8
Metrics: 5
Vulnerability scan completed. Report saved to vulnerability_report_20261007_113216.json
Summary: 3 vulnerabilities found
  Critical: 1
    High: 1
    Medium: 1
    Low: 0
    Unknown: 0
```

Fig 3.3a: Control Verification

```
(icdfa@icdfa)-[~/security_governance_lab]
$ cd ~/security_governance_lab
./simulate_attack.py
Simulating attack on vulnerable Struts application...
System is vulnerable to Struts S2-045 (CVE-2017-5638)
Simulating data exfiltration...
Simulating access to database...
Simulating exfiltration of customer data...
Attack simulation completed. Log saved to attack_log.json

(icdfa@icdfa)-[~/security_governance_lab]
$
```

Fig 3.3b: Post control attack Verification

Fig 3.3a and fig 3.3b: Verifies implemented governance controls, demonstrating security improvements while confirming that the Struts2 vulnerability remained exploitable and requires further remediation and continuous verification.

3.4 Governance Interpretation

The control implementation addressed several governance weaknesses identified in Part 2, including patch management, database security, monitoring, and segmentation and governance oversight. The results also highlighted the importance of validation. A control should not be considered effective simply because a script, policy or document exists; its actual operating state must be tested and evidenced. The remaining Struts vulnerability shows why critical remediation must be followed through to completion and independently verified.

Part 4: Developing Security Governance Metrics and Reporting

4.1 The security governance metrics script

```
(icdfa@icdfa)-[~/security_governance_lab]
$ cd ~/security_governance_lab
nano governance_metrics.py

(icdfa@icdfa)-[~/security_governance_lab]
$ cd ~/security_governance_lab
chmod +x governance_metrics.py
python3 -m py_compile governance_metrics.py

(icdfa@icdfa)-[~/security_governance_lab]
$ ./governance_metrics.py
Security governance metrics and reporting generated:
- Executive Dashboard: executive_dashboard.html
- Board Report: board_report.md
- Visualizations: metrics_visualizations/patch_compliance.png, metrics_visualizations/vuln_remediation.png, metrics_visualizations/security_incidents.png, metrics_visualizations/monitoring_coverage.png, metrics_visualizations/policy_compliance.png
```

Fig 4.1: security governance metrics file

Fig 4.1: Confirms successful generation of the executive dashboard, board report, and security governance metric visualizations.

[Kindly click the link here](#) to find the “**The security governance metrics script**”.

4.2 The executive dashboard HTML file

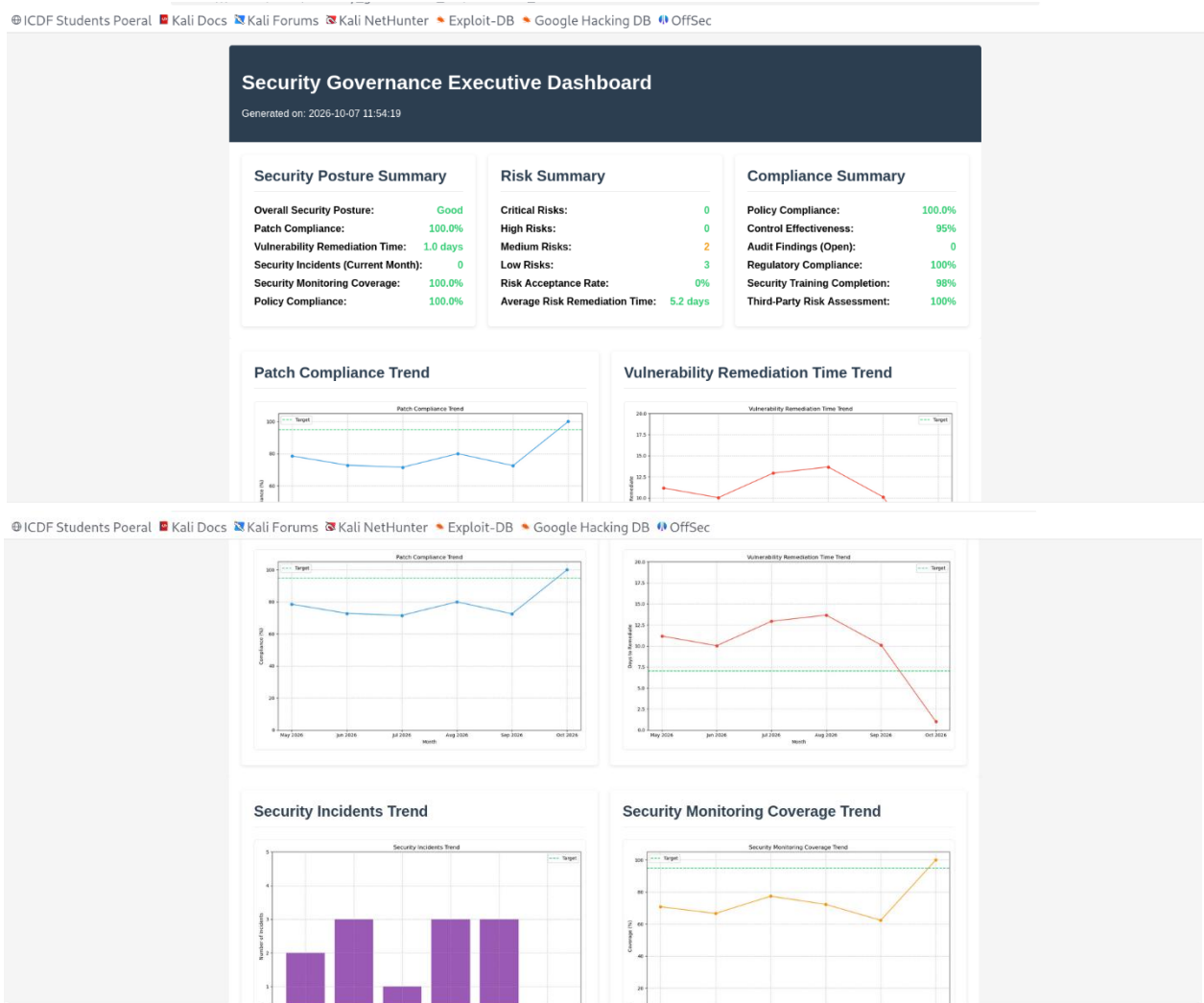


Fig 4.2: The executive dashboard

Fig 4.2: Displays the executive security governance dashboard summarizing current metrics, risks, compliance, trends, and recommendations.

[Kindly click the link here](#) to find the “**The executive dashboard HTML file**”

4.3 Board Report

[Kindly click the link here](#) to find the “**Board Report**”

The board report shows the board-level security governance report summarizing metrics, risks, compliance status, improvements, and recommendations.

4.4 The metrics framework document

```
(icdfa@icdfa)-[~/security_governance_lab]
$ cd ~/security_governance_lab
nano governance_metrics_framework.md

(icdfa@icdfa)-[~/security_governance_lab]
$ ls -l governance_metrics_framework.md
-rw-rw-r-- 1 icdfa icdfa 4438 Oct  7 11:57 governance_metrics_framework.md

(icdfa@icdfa)-[~/security_governance_lab]
$
```

Fig 4.4: The metrics framework document

Fig 4.4: Shows the documented framework defining security governance metrics, targets, data sources, reporting frequency, and ownership.

[Kindly click the link here](#) to find the “**Metrics Framework Document**”

4.5 The dashboard user guide

```
(icdfa@icdfa)-[~/security_governance_lab]
$ cd ~/security_governance_lab
nano dashboard_user_guide.md

(icdfa@icdfa)-[~/security_governance_lab]
$ ls -l dashboard_user_guide.md
-rw-rw-r-- 1 icdfa icdfa 3301 Oct  7 11:59 dashboard_user_guide.md

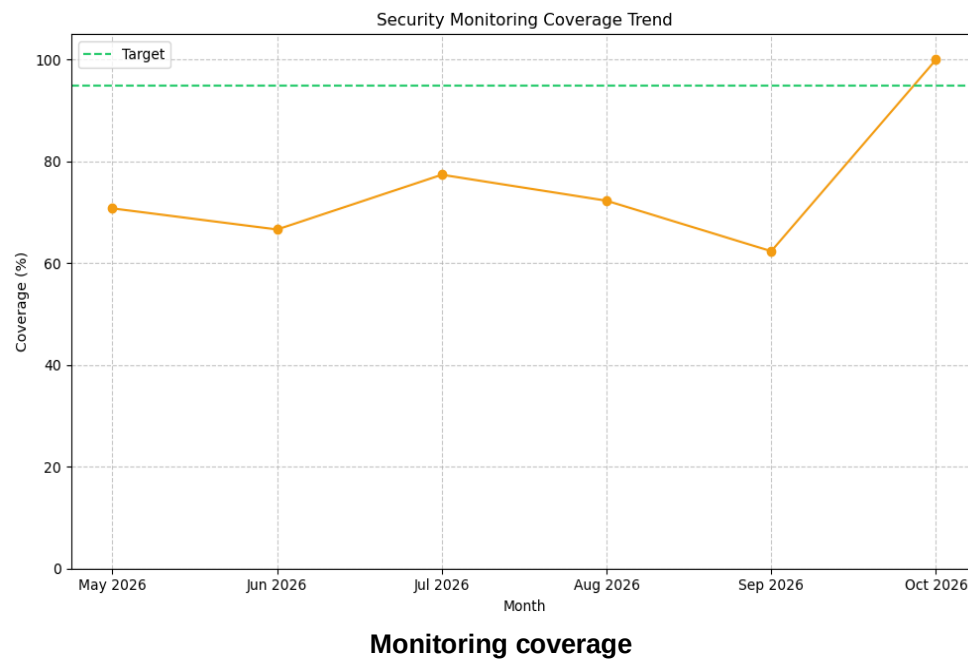
(icdfa@icdfa)-[~/security_governance_lab]
$
```

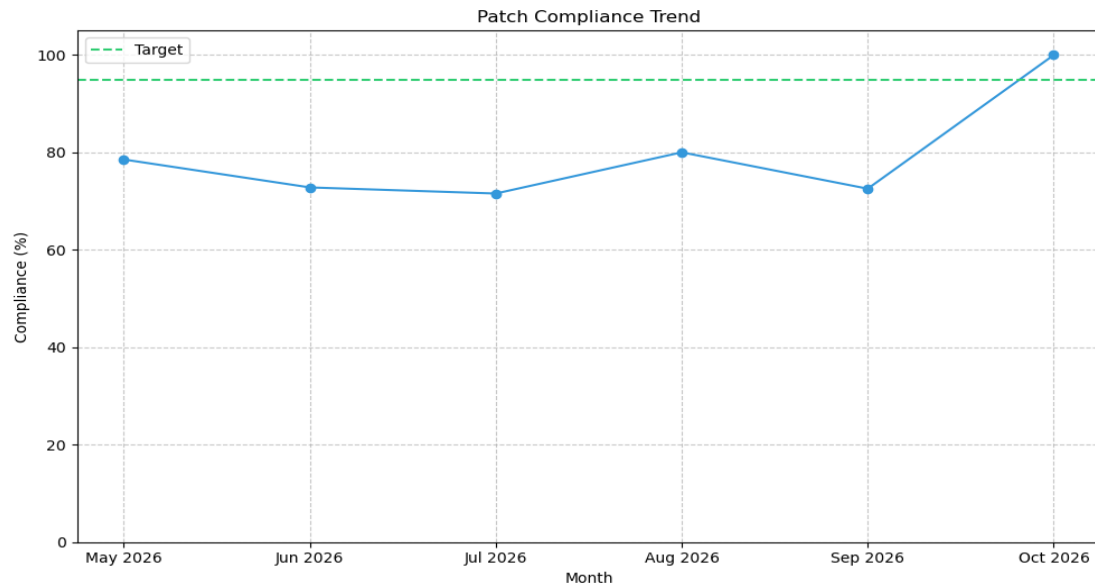
Fig 4.4: The dashboard user guide

Fig 4.5: Provides instructions for accessing, interpreting, refreshing, and using the security governance dashboard for decision-making.

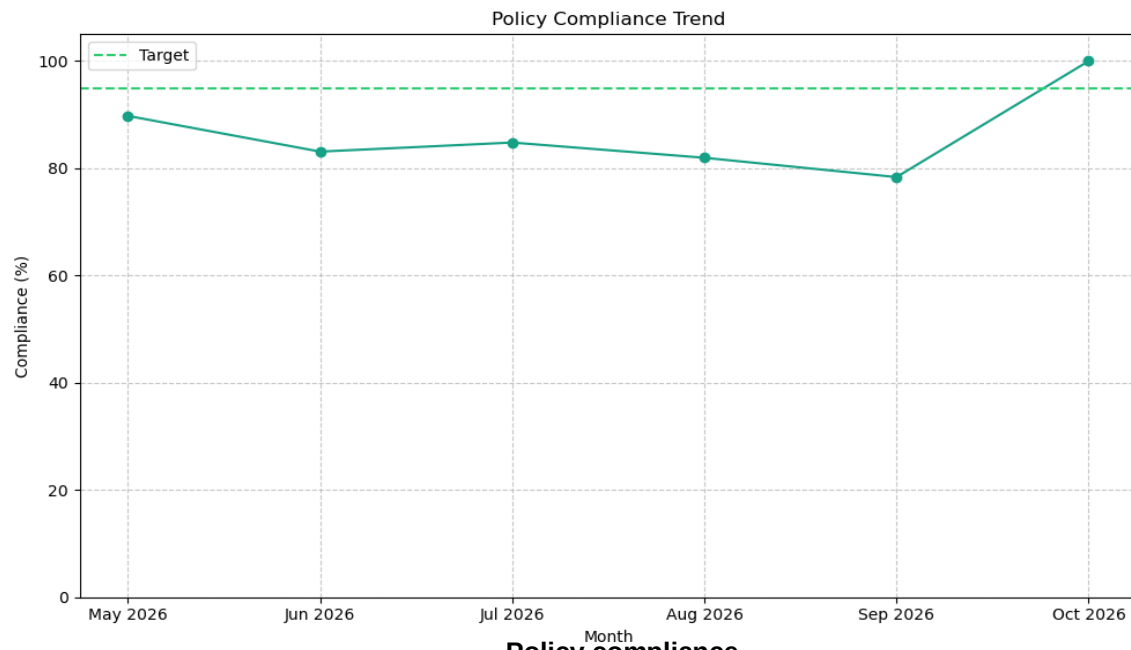
[Kindly click the link here](#) to find the **“The dashboard user guide”**

4.6 Screenshots of the dashboard and visualizations

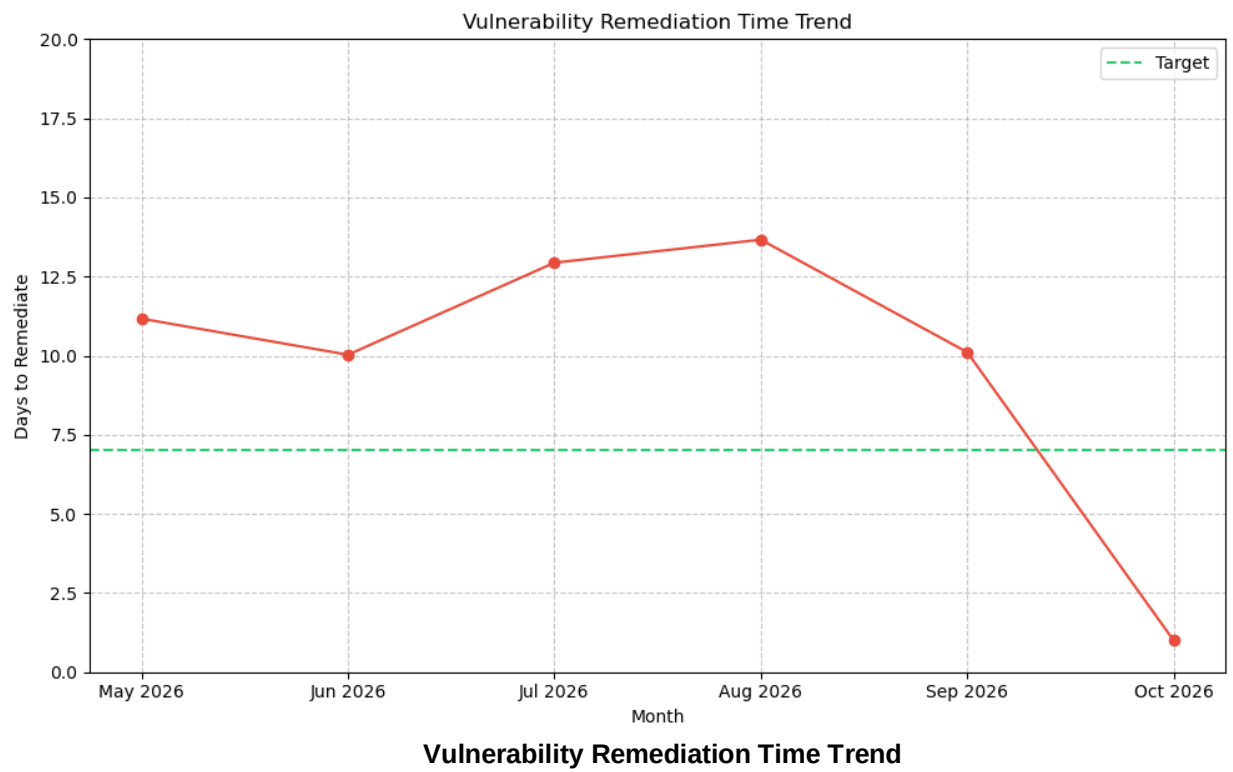
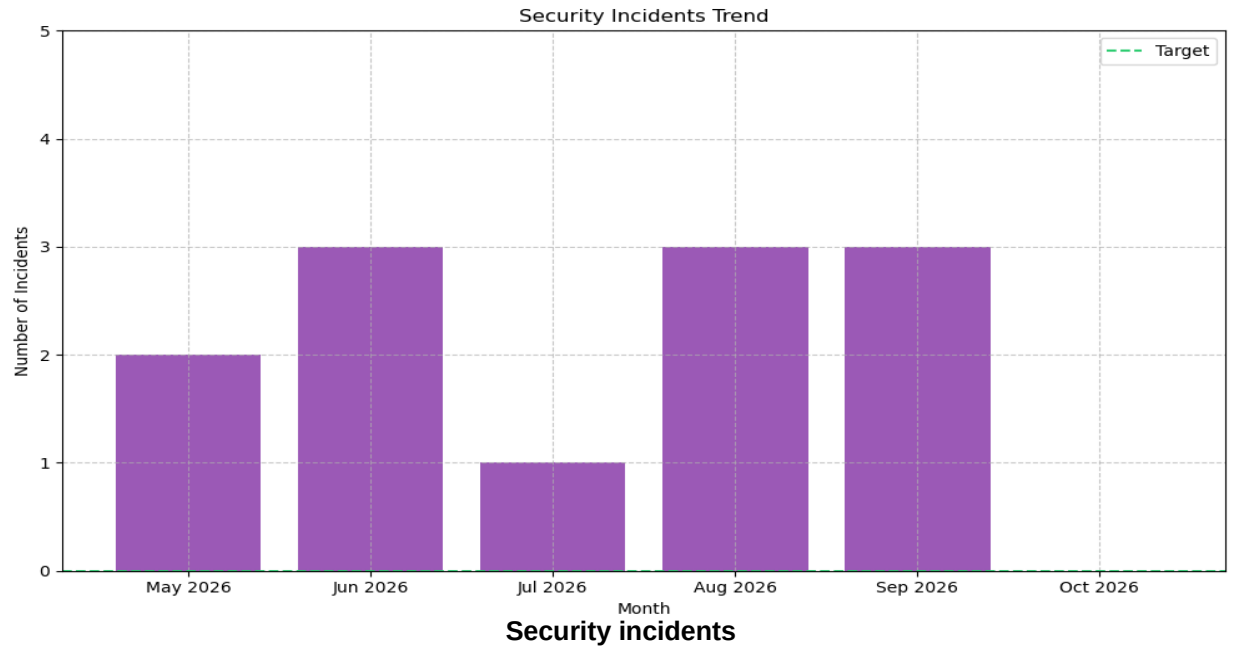




Patch compliance



Policy compliance



4.7 How Metrics and Reporting Support Governance

Security governance metrics and reporting help organizations understand whether security controls are working and whether security objectives are being achieved. Metrics such as patch compliance, vulnerability remediation time, security incidents, monitoring coverage and policy compliance provide management with measurable evidence instead of relying on assumptions. The executive dashboard presents this information in a form that senior management and board members can review quickly, while trend visualizations show whether performance is improving or getting worse over time.

Regular reporting also improves accountability because security teams can be given clear targets and responsibilities while management can track whether those targets are being achieved. Poor patch compliance or long remediation times can indicate that security risks are not being addressed quickly enough. Reporting these issues creates visibility and encourages corrective action. Overall, effective metrics and reporting turn security governance into a continuous process of measuring performance, identifying weaknesses, taking action and monitoring improvement. This supports better oversight and reduces the likelihood that known security weaknesses will be ignored.

Conclusion

The practical lab shows the full security governance cycle: establishing a controlled environment, identifying vulnerabilities and governance failures, implementing controls, validating those controls, and reporting security posture through governance metrics. The exercise reinforced that governance is not limited to policies. Effective governance requires ownership, timely remediation, measurable targets, monitoring, validation and management oversight.

An important finding from the lab was that control implementation did not automatically remove every risk. The post-control verification still showed the Struts

vulnerability and the authorized attack simulation remained successful. This evidence should be presented transparently because it demonstrates why continuous verification and corrective action are essential parts of effective security governance.

Challenges Encountered

- **Docker environment configuration:** Setting up the Docker environment required troubleshooting the external secgov_network and ensuring that the web server, database server, and monitoring container were connected correctly.
- **Struts2 vulnerable application:** The initial Docker configuration required correction to use the appropriate Vulhub Struts2 image and version. The configuration was updated to use **Struts2 2.3.30**, allowing the controlled vulnerability simulation to run successfully.
- **Vulnerability scanner compatibility:** The vulnerability scanner encountered an output/encoding issue when checking network connectivity in the Kali environment. A small adjustment was made to the Python subprocess handling so that the command output could be processed correctly. This did not change the purpose of the vulnerability scan.

Despite these challenges, the required security governance activities were completed successfully. The adjustments made were mainly environment and compatibility changes required to run the assessment on Kali Linux rather than Ubuntu. The core objectives, security controls, vulnerability analysis, governance metrics, and reporting requirements of the lab remained unchanged.